**Лабораторная работа №3.**
**Транспонирование матрицы и исследование локальности памяти.**

Выполнила: Колодезникова Софья, 11-400

### Характеристики системы

*Модель процессора: Core Ultra 5 125H*
*Число ядер: 14 (18 логических процессоров)*
*Размеры кешей: L1 = 1,4 МБ, L2 = 14 МБ, L3 = 18 МБ*
*Версия компилятора: GCC 16.2.0*

### Результаты экспериментов

Эксперимент 1. Влияние порядка обхода.

| N | Время rows, с | Время cols, с | BW rows, ГБ/с | BW cols, ГБ/с |
|---|---|---|---|---|
| 256 | 0,0002 | 0,0001 | 2,38 | 6,45 |
| 512 | 0,0009 | 0,0004 | 2,33 | 5,95 |
| 1024 | 0,0057 | 0,0043 | 1,48 | 1,97 |
| 2048 | 0,0294 | 0,0888 | 1,14 | 0,38 |
| 5000 | 0,1136 | 0,1991 | 1,76 | 1,00 |

*Для малых N взято среднее время по нескольким повторам внутри замера, иначе получаем 0,0000.*

Эксперимент 2. Влияние размера блока при *1 потоке, N = 5000*.

| Размер блока b | Время, с | BW, ГБ/с |
|---|---|---|
| 4 | 0,0346 | 5,79 |
| 8 | 0,0206 | 9,73 |
| 16 | 0,0197 | 10,17 |
| 32 | 0,0167 | 12,00 |
| 64 | 0,0201 | 9,94 |
| 128 | 0,0226 | 8,87 |

Эксперимент 3. Влияние количества потоков при $N = 5000$, $b = 32$, $T_1 = 0{,}1136$.

| Потоки | Время rows_omp, с | S rows_omp | Время blocked_omp, с | S blocked_omp |
|---|---|---|---|---|
| 1 | 0,1123 | 1,01 | 0,0162 | 7,01 |
| 2 | 0,0602 | 1,89 | 0,0126 | 9,02 |
| 4 | 0,0293 | 3,88 | 0,0079 | 14,38 |
| 8 | 0,0268 | 4,24 | 0,0072 | 15,78 |
| 16 | 0,0209 | 5,44 | 0,0059 | 19,25 |

Эксперимент 4. False sharing при *8 потоках, N = 5000*.

| Стратегия | Время, с | BW, ГБ/с |
|---|---|---|
| schedule(static) | 0,0284 | 7,03 |
| schedule(static, 1) | 0,0360 | 5,56 |
| schedule(static, 8) | 0,0306 | 6,55 |

*Дополнительное задание на 1 балл. Транспонирование на месте.*

| Стратегия | Потоки | Время, с | BW, ГБ/с |
|---|---|---|---|
| inplace | 1 | 0,0150 | 13,33 |
| inplace_omp | 8 | 0,0050 | 40,00 |
| blocked_omp | 8 | 0,0071 | 28,13 |

*Сравнение: транспонирование на месте при 8 потоках быстрее blocked_omp примерно в 1,4 раза, так как оно работает с одной матрицей, а не с двумя.*

### Вывод

В transpose_rows чтение A идёт подряд, а запись в B с шагом N, в transpose_cols наоборот. Каждое обращение с шагом N затрагивает новую кеш-линию и использует из неё всего 4 байта, поэтому, пока матрицы помещаются в кеш, это не мешает, а потом приводит к потерям из-за вытеснения линий. На малых N, когда обе матрицы помещаются в L3, версии близки. Заметной разница становится от N = 2048, когда обе матрицы больше L3.

Блочная версия при b* = 32 примерно в 7 раз быстрее rows, потому что блок обрабатывается целиком в кеше, а два блока 32×32 типа float меньше L1-кеша данных одного ядра. При b = 64 пара блоков примерно равно L1, а при b = 128 уже не помещается, поэтому время растёт.

Транспонирование не содержит вычислений, только чтение и запись, поэтому потоки делят одну пропускную способность памяти: время blocked_omp при 16 потоках упало лишь с 0,0162 до 0,0059 с, тогда как в ЛР2, где каждый член суммы требует вычисления sin и sqrt, ускорение на 16 потоках составило 4,4.

При schedule(static, 1) соседние строки A достаются разным потокам, а в B они записывают соседние элементы одной кеш-линии, поэтому линия постоянно пересылается между ядрами → время растет с 0,0284 до 0,0360 с. При schedule(static) потоки получают непрерывные диапазоны строк, поэтому пересечений почти нет, а при schedule(static, 8) порция из 8 элементов занимает половину линии, поэтому штраф меньше (0,0306 с).